DEEP LEARNING

Unit I
Introduction to Neural Network

Introduction to Neural Networks

Neural networks form the foundation of deep learning, a subfield of machine learning that focuses on models with multiple layers capable of learning hierarchical representations of data. Deep learning aims to replicate the human brain's ability to process information, recognize patterns, and make decisions. Neural networks achieve this by leveraging large datasets, computational power, and sophisticated architectures.

What is a Neural Network?

A neural network is a computational model inspired by the structure and function of biological neural networks. It consists of interconnected layers of nodes (neurons) that process data and extract features. Neural networks learn by adjusting their parameters (weights and biases) to minimize errors in predictions or outputs.

Why Deep Learning?

Deep learning excels in handling unstructured data, such as images, audio, and text, where traditional machine learning approaches struggle. Neural networks in deep learning are designed to automatically extract features from raw data, eliminating the need for manual feature engineering.

Key Concepts in Deep Learning Neural Networks:

- **Deep Architectures**:
  - Neural networks in deep learning often have many layers (hence "deep").
  - These layers learn hierarchical representations, with lower layers capturing simple features (e.g., edges in an image) and higher layers capturing more abstract features (e.g., shapes or objects).
- **Forward Propagation**:
  - The process of passing data through the network from input to output.
  - Each layer transforms the data and passes it to the next layer.
- **Loss Function**:
  - Quantifies the difference between predicted and actual values.
  - Examples: Mean Squared Error (MSE) for regression and Cross-Entropy Loss for classification.
- **Backpropagation and Optimization**:
  - Backpropagation computes gradients of the loss function with respect to weights and biases.
  - Optimization algorithms like Stochastic Gradient Descent (SGD) or Adam adjust these parameters to minimize loss.

Types of Neural Networks in Deep Learning:

- **Convolutional Neural Networks (CNNs)**:
  - Used for image-related tasks.
  - Employ convolutional layers to detect spatial patterns.
- **Recurrent Neural Networks (RNNs)**:
  - Process sequential data like time series and text.
  - Variants include LSTMs (Long Short-Term Memory) and GRUs (Gated Recurrent Units).
- **Transformer Networks**:
  - Revolutionized natural language processing (e.g., GPT, BERT).
  - Use self-attention mechanisms to process sequences efficiently.
- **Autoencoders**:
  - Used for unsupervised learning and data compression.
  - Learn to encode data into a compressed representation and decode it back.
- **Generative Adversarial Networks (GANs)**:
  - Generate realistic synthetic data.
  - Involve two networks: a generator and a discriminator.

Applications of Neural Networks in Deep Learning:

- **Computer Vision**:
  - Image classification, object detection, and medical imaging.
- **Natural Language Processing (NLP)**:
  - Machine translation, sentiment analysis, and text summarization.
- **Speech and Audio Processing**:
  - Voice recognition, music generation, and audio analysis.
- **Autonomous Systems**:
  - Self-driving cars and robotics.
- **Healthcare**:
  - Disease diagnosis, drug discovery, and personalized treatment plans.
- **Gaming**:
  - AI for game-playing and procedural content generation.

Advantages of Neural Networks in Deep Learning:

- **Feature Extraction:** Automatically learns features from raw data.
- **Scalability:** Handles large datasets and complex problems.
- **Accuracy:** Achieves state-of-the-art results in many domains.

Neural networks in deep learning represent a paradigm shift in artificial intelligence, enabling machines to solve problems previously considered unsolvable. By understanding their structure and operation, we can leverage their power for innovation across diverse fields.

Neural Networks and types

Neural Networks:

- With the advancements of artificial intelligence and machine learning, neural networks are becoming more widely discussed thanks to their role in deep learning.
- Neural networks learn continuously and, as a result, can improve over time, making intelligent decisions based on the insights identified within the data. Many industries benefit from using neural networks with applications, including medical diagnostics, energy demand forecasting, targeted marketing, and financial prediction.

An introduction to artificial intelligence, machine learning, and deep learning:

Before exploring neural networks, it's important to understand artificial intelligence, machine learning, and deep learning and how they are related.

- Artificial intelligence describes the process of computers being trained to mimic the human brain in how it learns and solves problems. Computers can do this through different types of learning: machine learning and deep learning.
- The term "artificial intelligence" can be traced back to 1956 when computer scientist John McCarthy coined it. However, in 1950, British mathematician and computer scientist Alan Turing discussed the concept of machines being able to think in a groundbreaking paper that played a significant role in the development of artificial intelligence [1].

An introduction to artificial intelligence, machine learning, and deep learning:

- Machine learning is a series of algorithms, each taking in information, analysing it, and using that insight to make an informed decision. As machine learning algorithms are given more data, they can become increasingly intelligent and make better, more informed decisions.
- Deep learning is a subset of machine learning. Neural networks play a role in deep learning, as they allow data to be processed without a human pre-determining the program. Instead, neural networks communicate data with one another similarly to how the brain functions, creating a more autonomous process.

Overview of neural networks

- The basic structure of a neural network consists of three main components: the input layer, the hidden layer, and the output layer. A neural network can have one or multiple input, hidden, or output layers depending on complexity.
- Information is received in the input layer, and the input node processes it,decides how to categorise it, and transfers it to the next layer: the hidden layer.
- Information is received from both the input layer and other hidden layers in the hidden layer. There are various hidden layers based on the type of neural network being used. At this point in the process, hidden layers take the input, process the information from the previous layer, and then move it on to the next layer, either another hidden layer or the output layer.
- The output layer is the final layer in a neural network. After receiving the data from the hidden layer (or layers), the output layer processes it and produces the output value.

Types of Neural Networks

8 types of neural networks:

Various neural networks exist, each with a unique structure and function. This list will discuss eight commonly used neural networks in today's technology.

1. Convolutional neural networks:

Convolutional neural networks (CNNs) can input images, identify the objects in a picture, and differentiate them from one another. Their real-world applications include pattern recognition, image recognition, and object detection. A CNN's structure consists of three main layers. First is the convolutional layer, where most of the computation occurs. Second is the pooling layer, where the number of parameters in the input is reduced. Lastly, the fully connected layer classifies the features extracted from the previous layers.

2. Recurrent neural networks:

Recurrent neural networks (RNNs) can translate language, speech recognition, natural language processing, and image captioning. Examples of products using RNNs include smart home technologies and voice command features on mobile phones. Feedback loops in the structure of RNNs allow information to be stored similarly to how your memory works.

Types of Neural Networks

3. Radial basis functions networks:

Radial basis function (RBF) networks differ from other neural networks because the input layer performs no computations. Instead, it passes the data directly to the hidden layer. As a result, RBFs have a faster learning speed. Applications of RBF networks include time series prediction and function approximation.

4. Long short-term memory networks:

Long short-term memory (LSTM) networks are unique and can sort data into short-term and long-term memory cells depending on whether or not the data needs to be looped back into the network as data points or entire sequences. LSTM can also be used in handwriting recognition and video-to-text conversion.

5. Multilayer perceptrons:

Multilayer perceptrons (MLPs) are a neural network capable of learning the relationship between linear and non-linear data. Through backpropagation, MLPs can reduce error rates. Applications that benefit from MLPs include face recognition and computer vision.

Types of Neural Networks

6. Generative adversarial networks:

Generative adversarial networks (GANs) can generate new data sets that share the same statistics as the training set and often pass as actual data. An example of this you've likely seen is art created with AI. GANs can replicate popular art forms based on patterns in the training set, creating pieces often indistinguishable from human artwork.

7. Deep belief networks:

Deep belief networks (DBNs) are unique because they stack individual networks that can use each other's hidden network layers as the input for the next layer. This allows for the neural networks to be trained faster. They are used to generate images and motion-capture data.

8. Self-organising maps:

Self-organising maps (SOMs), or Kohonen maps, can transform extensive complex data sets into understandable two-dimensional maps where geometric relationships can be visualised. This can happen because SOMs use competitive learning algorithms in which neurons must compete to be represented in the output. This is decided by which neurons best represent the input. Practical applications of SOMs include displaying voting trends for analysis and organising complex data collected by astronomers so it can be interpreted.

Gradient Descent

Gradient Descent is a fundamental optimization algorithm widely used in deep learning to minimize the error (or cost) of a model by updating its parameters (weights and biases). It works by iteratively adjusting the parameters in the opposite direction of the gradient of the cost function with respect to the parameters. This adjustment aims to find the optimal set of parameters that minimize the cost function.

Key Concepts of Gradient Descent:

- **Cost Function**:
  - Measures the error between the predicted output and the actual target.
  - Examples include Mean Squared Error (MSE) for regression and Cross-Entropy Loss for classification.
- **Gradient**:
  - The derivative of the cost function with respect to the model parameters.
  - Indicates the direction and magnitude of the steepest ascent. To minimize the cost, we move in the opposite direction.
- **Learning Rate** (η\etaη):
  - A hyperparameter that controls the size of the steps taken toward the minimum.
  - If too large, it might overshoot the minimum; if too small, convergence can be slow.

Gradient Descent

Types of Gradient Descent:

- **Batch Gradient Descent**:
  - Uses the entire training dataset to compute the gradient.
  - Pros: Stable convergence.
  - Cons: Computationally expensive for large datasets.
- **Stochastic Gradient Descent (SGD)**:
  - Uses a single training example to compute the gradient for each update.
  - Pros: Faster updates, good for online learning.
  - Cons: Noisy updates can make convergence less stable.
- **Mini-Batch Gradient Descent**:
  - Uses a subset (mini-batch) of the training data to compute the gradient.
  - Pros: Balances efficiency and stability.
  - Cons: Requires tuning of batch size.

Batch Gradient Descent

Batch Gradient Descent entails computation (involved in each step of gradient descent) over the entire training set at each step and hence it is highly slow on very big training sets. As a result, Batch Gradient Descent becomes extremely computationally expensive. This is ideal for error manifolds that are convex or somewhat smooth. Batch Gradient Descent also scales nicely as the number of features grows. Batch gradient descent has several advantages and disadvantages, including:

Advantages:

- **Convergence stability**: Batch gradient descent provides a smoother path to convergence, which can help with reliable model training.
- **Global minimum**: Batch gradient descent can converge to the global minimum cost function under certain conditions.
- **Large datasets**: Batch gradient descent can work well on large datasets.
- **Convex functions**: Batch gradient descent is excellent for convex functions.
- **Easy implementation**: Batch gradient descent is easy to implement.

Disadvantages:

- **Local minima**: Batch gradient descent can be difficult to escape local minima.
- **Larger datasets**: Batch gradient descent can be impractical for larger datasets.
- **Slow convergence**: Batch gradient descent can be slow to converge.
- **Noisy**: Batch gradient descent can be noisy, which can lead to less stability.

Stochastic Gradient Descent

Stochastic Gradient Descent (SGD) is a variant of the Gradient Descent algorithm used for optimizing machine learning models. In this variant, only one random training example is used to calculate the gradient and update the parameters at each iteration. Here are some of the advantages and disadvantages of using SGD:

Advantages of Stochastic Gradient Descent:

- **Speed:** SGD is faster than other variants of Gradient Descent such as Batch Gradient Descent and Mini-Batch Gradient Descent since it uses only one example to update the parameters.
- **Memory Efficiency:** Since SGD updates the parameters for each training example one at a time, it is memory-efficient and can handle large datasets that cannot fit into memory.
- **Avoidance of Local Minima:** Due to the noisy updates in SGD, it has the ability to escape from local minima and converges to a global minimum.

Stochastic Gradient Descent

Disadvantages of Gradient Descent:

- **Noisy updates**: The updates in SGD are noisy and have a high variance, which can make the optimization process less stable and lead to oscillations around the minimum.
- **Slow Convergence:** SGD may require more iterations to converge to the minimum since it updates the parameters for each training example one at a time.
- **Sensitivity to Learning Rate:** The choice of learning rate can be critical in SGD since using a high learning rate can cause the algorithm to overshoot the minimum, while a low learning rate can make the algorithm converge slowly.
- **Less Accurate:** Due to the noisy updates, SGD may not converge to the exact global minimum and can result in a suboptimal solution. This can be mitigated by using techniques such as learning rate scheduling and momentum-based updates.

Mini-Batch Gradient Descent

Mini-batch gradient descent is an optimization method that updates neural network weights using a subset of training data. It has several advantages, including faster convergence and memory efficiency, but it also has some disadvantages.

Advantages:

➢ **Faster convergence:**

- Mini-batch gradient descent updates parameters more frequently than batch gradient descent, which can lead to faster convergence.

➢ **Memory efficiency:**

- Mini-batches allow training on large datasets without loading the entire dataset into memory at once.

➢ **Smoother gradient estimation:**

- Mini-batch gradient descent offers a smoother path toward convergence than stochastic gradient descent.

➢ **Efficient parallel processing:**

- Mini-batch gradient descent can be efficiently processed in parallel.

Mini-Batch Gradient Descent

Disadvantages:

- **Learning rate and batch size selection:**
  - The choice of learning rate and mini-batch size can impact the performance of mini-batch gradient descent.
- **Additional hyperparameter tuning:**
  - Mini-batch gradient descent requires additional hyperparameter tuning.
- **Increased complexity:**
  - Mini-batch gradient descent can be more complex to implement than other methods.
- **May not converge to a global minimum:**
  - If the batch size is not well-tuned, mini-batch gradient descent may not converge to a global minimum.
  - Mini-batch gradient descent is a widely used method for training deep learning models.

Gradient Descent

Variants of Gradient Descent:

- **Momentum**:

Adds a fraction of the previous update to the current update to accelerate convergence and smooth oscillations.

- **Adagrad**:

Adapts the learning rate for each parameter based on past gradients, making larger updates for infrequently updated parameters.

- **RMSProp**:

Uses a moving average of squared gradients to normalize updates, preventing large oscillations.

- **Adam (Adaptive Moment Estimation)**:

Combines Momentum and RMSProp to adapt learning rates for each parameter, resulting in faster and more stable convergence.

Gradient Descent

Gradient Descent in Deep Learning:

1. **Backpropagation**:
   - Gradient descent is used alongside backpropagation to calculate gradients of the cost function with respect to each parameter in the network.
   - Gradients are computed using the chain rule across multiple layers.
2. **Challenges**:
   - **Vanishing/Exploding Gradients**: Gradients can become very small or very large in deep networks, making learning difficult.
   - **Local Minima and Saddle Points**: May trap the optimization process.
3. **Solutions**:
   - Proper initialization techniques (e.g., Xavier, He initialization).
   - Advanced optimizers (e.g., Adam).
   - Batch normalization to stabilize training.

Gradient Descent

Practical Tips:

- **Learning Rate Tuning**:
  - Use a learning rate schedule (e.g., reduce learning rate as training progresses).
  - Use learning rate warm-up in the initial stages.
- **Regularization**:
  - Techniques like dropout and L2 regularization help prevent overfitting.
- **Early Stopping**:
  - Monitor validation loss to terminate training when the model starts to overfit.

Gradient Descent remains a cornerstone of deep learning, driving the optimization of neural networks efficiently when paired with computational advancements and algorithmic innovations.

Training Neural Networks

- Training a neural network involves the process of adjusting the weights and biases of the network to minimize the difference between the predicted outputs and the actual outputs for a given set of training data.

- This process is typically done through a combination of forward propagation and backpropagation.

- This process is typically done through a combination of forward propagation and backpropagation. Here's a step-by-step overview of the training process:

Training Neural Networks

On each layer we learn different representation that gets more complicated with later hidden layers.Below is an example of a 3-layers neural network (we don't count input layer):

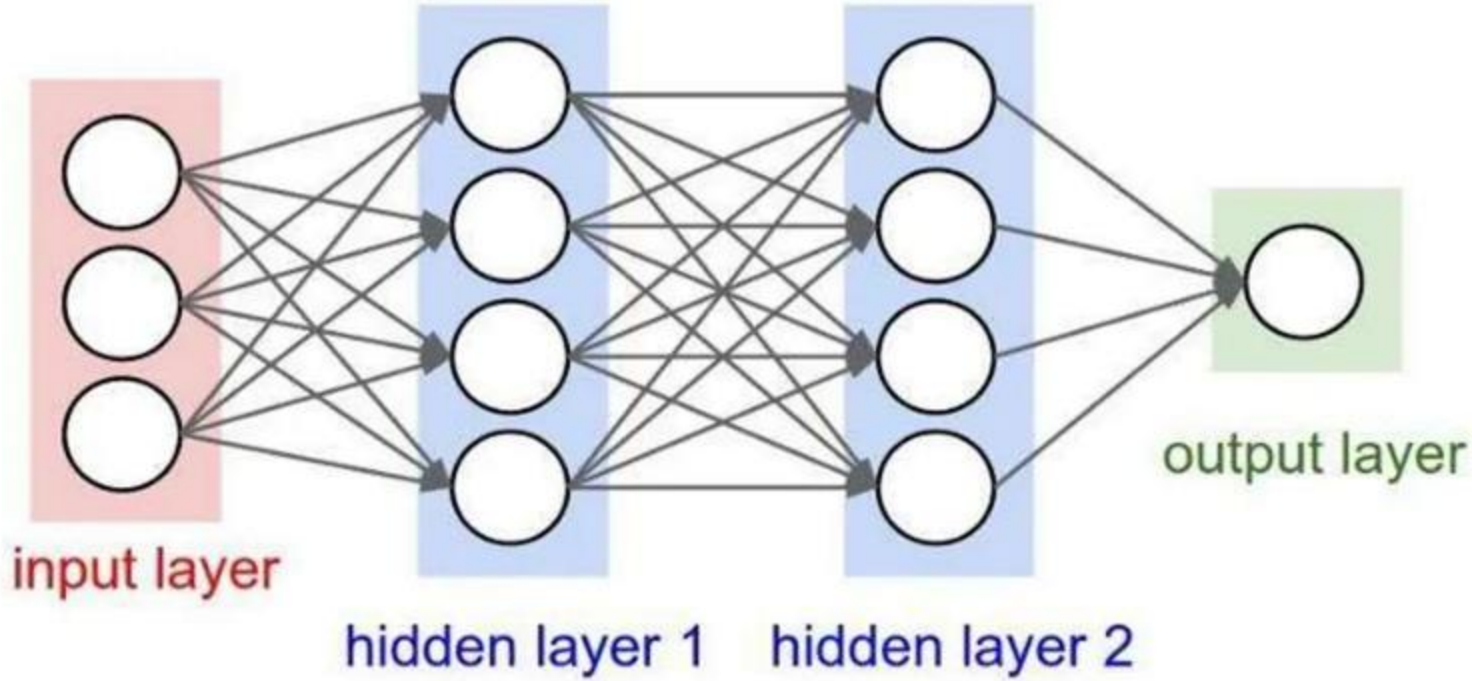
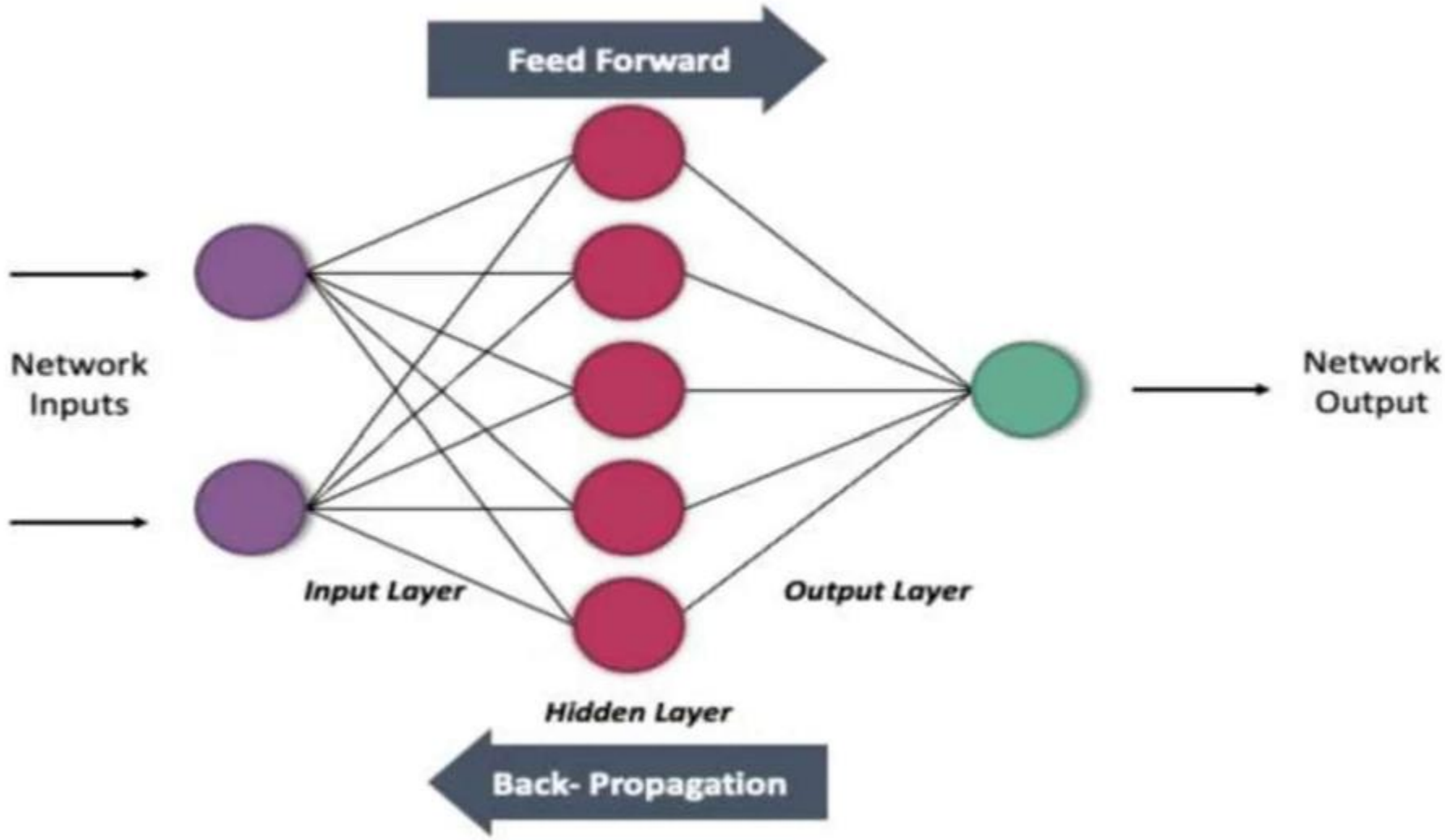
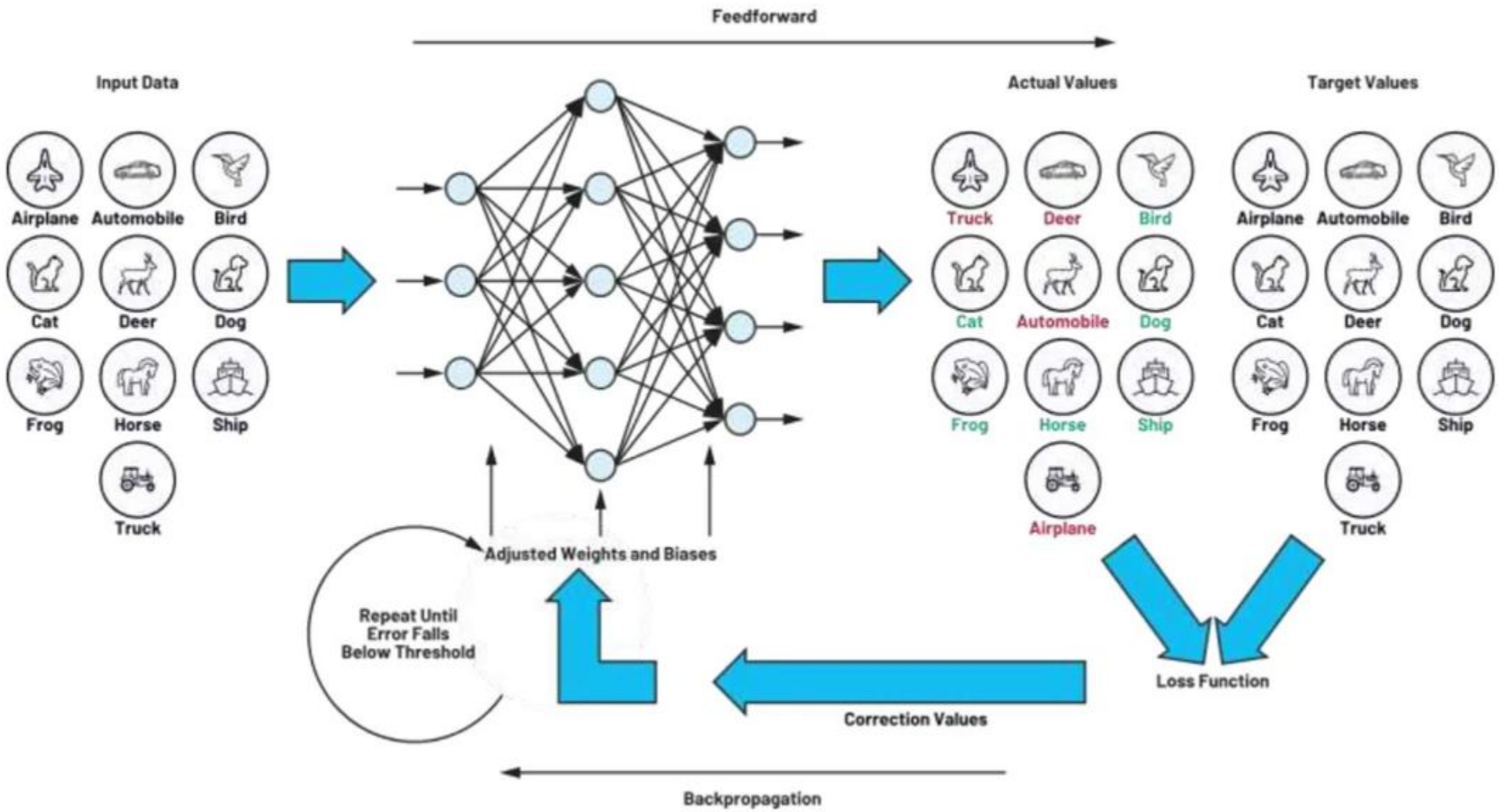


Figure 1: Neural Network with two hidden layers

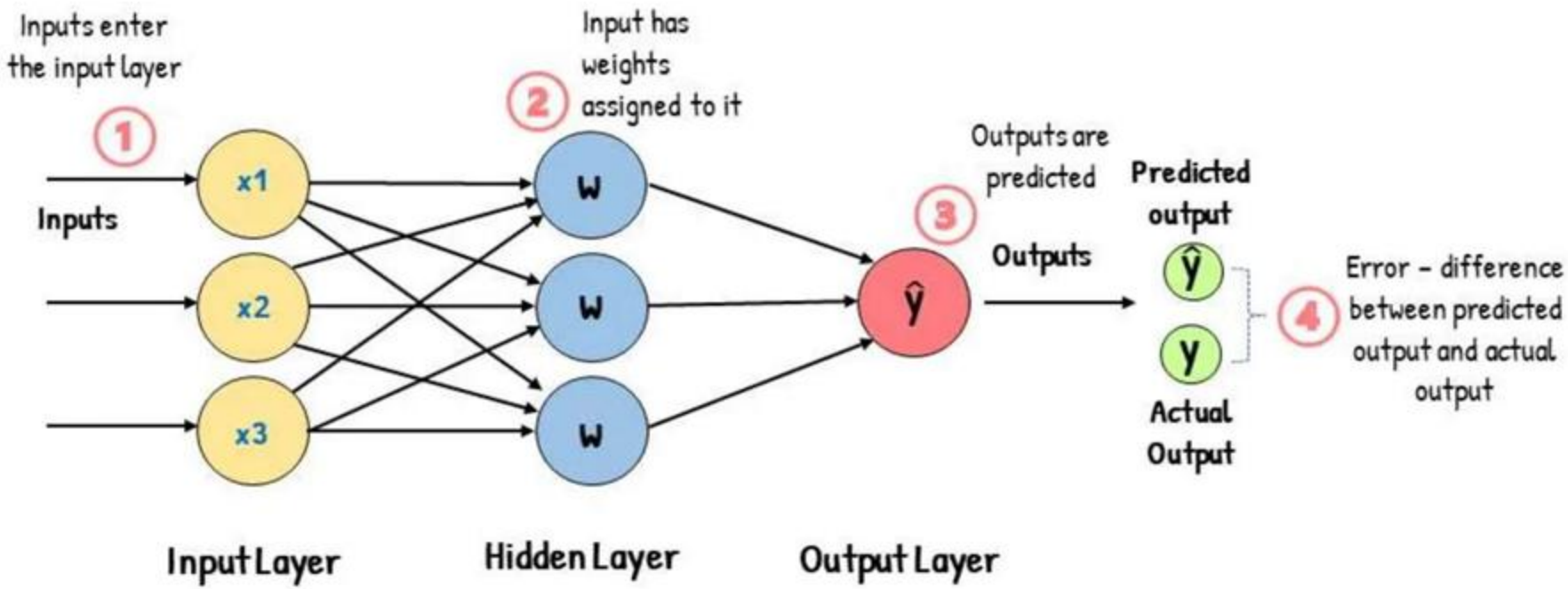
Training Neural Networks



Training Neural Networks



Feed- Forward Neural Network



Feed- Forward Neural Network

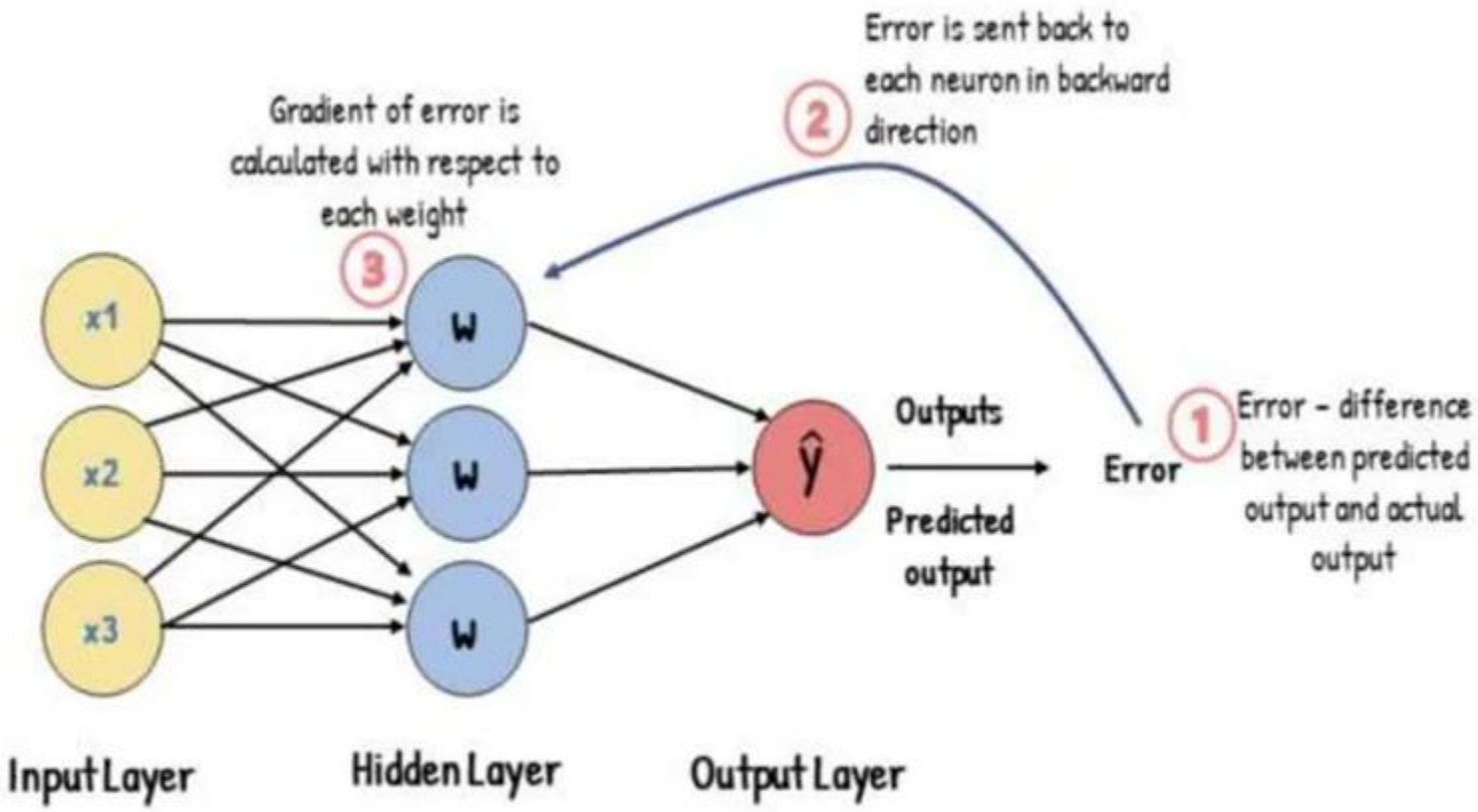
Forward Propagation:

- Input a training sample into the neural network.
- Propagate the input forward through the network to compute the predicted output.
- Apply activation functions at each neuron to introduce non-linearity.

Compute Loss:

- Compare the predicted output with the actual output using a loss function.
- The loss function quantifies the error between the predicted and actual outputs.

Backpropagation



Backpropagation

Backward Pass:

Compute the gradient of the loss with respect to the output of the neural network.

∂Loss / ∂Output

Backpropagate Errors:

- Propagate the gradient backward through the network to compute the gradients of the loss with respect to the weights and biases.
- Use the chain rule to calculate the contribution of each weight and bias to the overall loss.

Backpropagation

Update Weights and Biases:

- Adjust the weights and biases to minimize the loss by descending along the negative gradient direction.
- Update each weight and bias using the gradient and a chosen optimization algorithm (e.g., stochastic gradient descent)

New Weight=Old Weight−Learning Rate× ∂Weight / ∂Loss

Repeat for Each Training Sample or Batch:

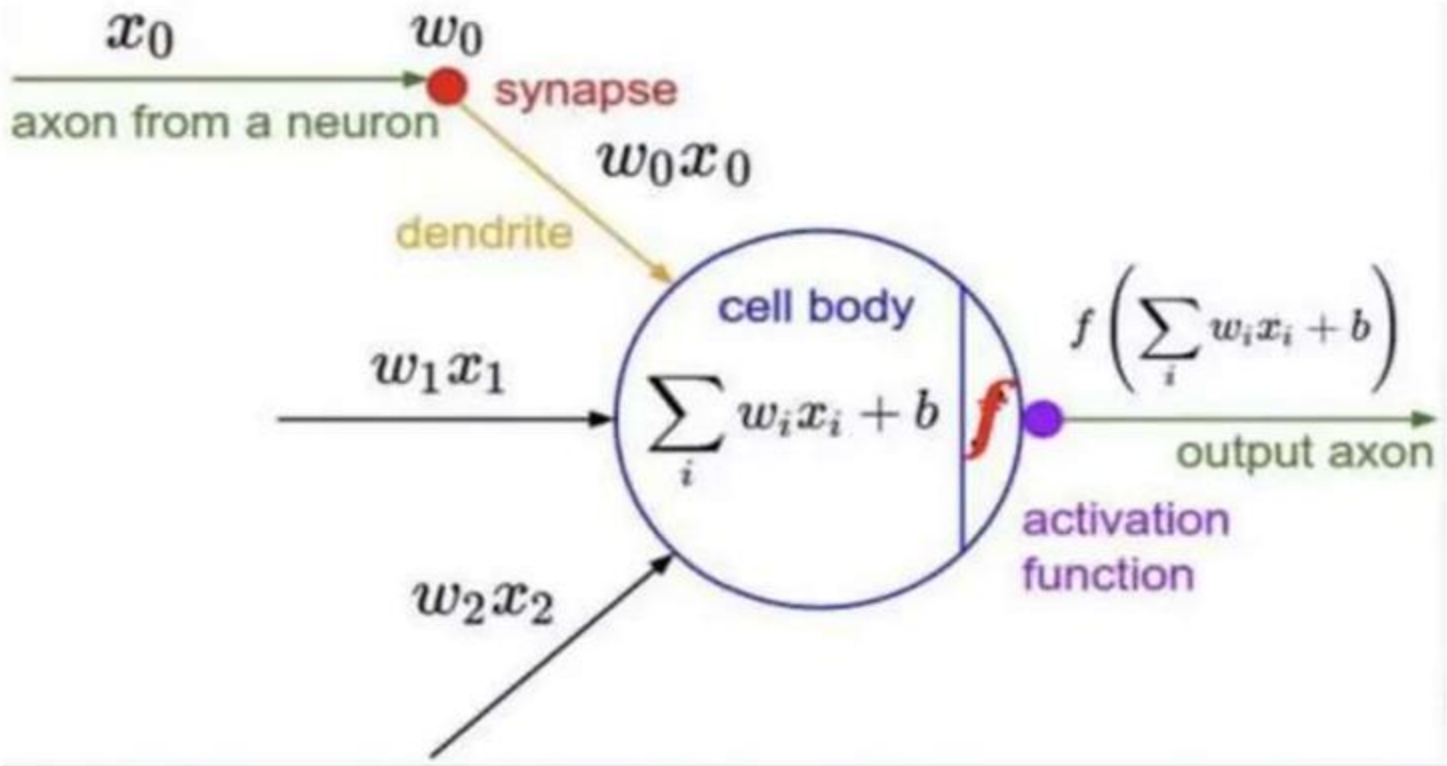
- Iterate through all training samples or batches, performing steps 1-5 for each.
- The algorithm can use full-batch, mini-batch, or stochastic gradient descent.

Repeat for Multiple Epochs:

- Continue the forward and backward passes for a specified number of epochs or until the loss converges.

Activation Function

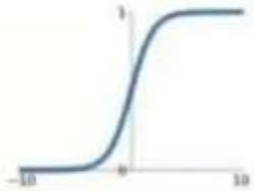
- Importance of Activation function is to introduce **non-linearity** into the network



Activation Function

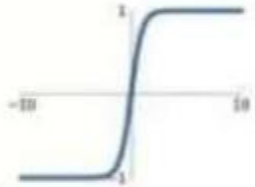
Sigmoid

$\sigma(x) = \frac{1}{1+e^{-x}}$



tanh

$\tanh(x)$



ReLU

$\max(0, x)$

Leaky ReLU

$\max(0.1x, x)$

Maxout

$\max(w_1^T x + b_1, w_2^T x + b_2)$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

For Output layer

- Sigmoid
- Tanh
- Softmax

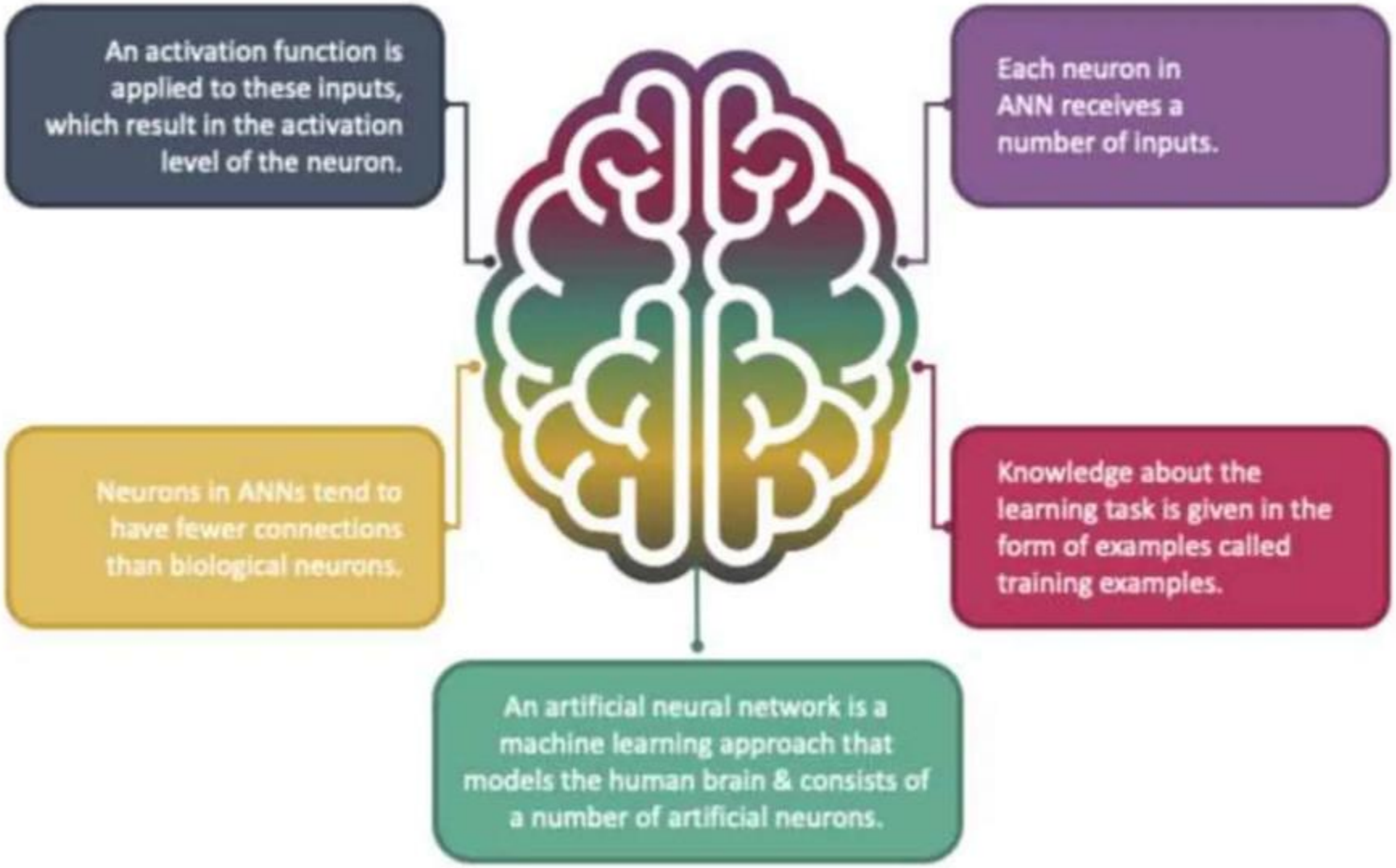
For Hidden Layer

- ReLU
- Leaky ReLU
- ELU

Activation Function

Name	Plot	Function, $f(x)$	Derivative of f, $f'(x)$	Range
Identity		x	1	$(-\infty, \infty)$
Binary step		$\begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$	$\begin{cases} 0 & \text{if } x \neq 0 \\ \text{undefined} & \text{if } x = 0 \end{cases}$	$\{0, 1\}$
Logistic, sigmoid, or soft step		$\sigma(x) = \frac{1}{1+e^{-x}}$[1]	$f(x)(1-f(x))$	$(0, 1)$
tanh		$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	$1 - f(x)^2$	$(-1, 1)$
Rectified linear unit (ReLU)[11]		$\begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases}$ $= \max\{0, x\} = x\mathbf{1}_{x>0}$	$\begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x > 0 \\ \text{undefined} & \text{if } x = 0 \end{cases}$	$[0, \infty)$

Activation Function



Sentiment Analysis

Sentiment Analysis is a significant application of Natural Language Processing (NLP) that involves determining the sentiment or emotion expressed in a piece of text. It is widely used in domains like social media monitoring, customer feedback analysis, and product reviews. Deep learning has revolutionized sentiment analysis by enabling models to capture complex patterns and nuances in text data. Below is an overview of sentiment analysis in the context of deep learning:

1. Overview of Sentiment Analysis:

Definition: Sentiment analysis identifies the sentiment of a text (positive, negative, neutral, or more granular emotional categories like joy, anger, sadness, etc.).

Applications:

- Customer feedback analysis
- Brand monitoring
- Movie/product review classification
- Political sentiment tracking

2. Role of Deep Learning in Sentiment Analysis:

- Deep learning has significantly improved sentiment analysis by providing models capable of understanding context, semantics, and syntactic structure.

➢ **Why Deep Learning?**

- Traditional machine learning models (e.g., SVM, Naïve Bayes) rely heavily on manual feature extraction and may fail to capture complex dependencies.
- Deep learning models like Recurrent Neural Networks (RNNs) and Transformers automatically learn feature representations, improving performance.

Sentiment Analysis

- **3. Deep Learning Models for Sentiment Analysis:**
- **a. Feedforward Neural Networks (FNNs):**
- Suitable for simple sentiment analysis tasks.
- Requires manually engineered features like bag-of-words or TF-IDF.
- **b. Recurrent Neural Networks (RNNs):**
- Captures sequential information in text.
- Popular variants include:
  - **LSTM (Long Short-Term Memory):** Overcomes vanishing gradient problems in traditional RNNs.
  - **GRU (Gated Recurrent Units):** Similar to LSTMs but computationally less expensive.
- **c. Convolutional Neural Networks (CNNs):**
- Extracts local features from text (e.g., phrases).
- Often combined with word embeddings like Word2Vec or GloVe for better representations.
- **d. Hybrid Models:**
- Combines CNNs and RNNs to leverage both local features (CNNs) and sequential dependencies (RNNs).
- **e. Transformers:**
- Dominates modern sentiment analysis.
- **BERT (Bidirectional Encoder Representations from Transformers):**
  - Captures bidirectional context in text.
  - Fine-tuned for specific sentiment analysis tasks.
- **GPT (Generative Pre-trained Transformer):**
  - Excels in generating human-like text but can also be fine-tuned for sentiment analysis.
- **Other Models:** RoBERTa, DistilBERT, and ALBERT

Sentiment Analysis

4. Preprocessing Steps:

- **Text Cleaning:** Remove unnecessary characters, HTML tags, etc.
- **Tokenization:** Split text into words, subwords, or characters.
- **Stop Word Removal:** Optional step depending on the model.
- **Word Embedding:** Convert tokens into dense vector representations (e.g., Word2Vec, GloVe, FastText, or Transformer embeddings).

5. Datasets for Sentiment Analysis:

- **IMDB Movie Reviews:** Contains positive and negative movie reviews.
- **Twitter Sentiment Analysis Dataset:** Focuses on tweets with sentiments.
- **Amazon Reviews:** Annotated reviews for various products.
- **Stanford Sentiment Treebank (SST):** Includes fine-grained sentiment labels for phrases.

6. Challenges:

- **Context Understanding:** Sarcasm and idiomatic expressions are hard to detect.
- **Domain Adaptation:** Models trained on one dataset may not generalize well to another domain.
- **Multilingual Sentiment Analysis:** Requires cross-lingual capabilities.
- **Imbalanced Data:** Sentiment datasets often have skewed distributions.

Sentiment Analysis

7. Tools and Libraries:

- **TensorFlow and Keras:** Frameworks for building deep learning models.
- **PyTorch:** Another popular framework with extensive NLP support.
- **Hugging Face Transformers:** Pre-trained transformer models like BERT, GPT, and RoBERTa.
- **NLTK and SpaCy:** Useful for preprocessing.

8. Evaluation Metrics:

- **Accuracy:** Proportion of correct predictions.
- **Precision, Recall, F1-Score:** Especially important for imbalanced datasets.
- **ROC-AUC:** For binary sentiment classification.

9. Future Trends:

- **Zero-shot Learning:** Models like GPT-3 and ChatGPT can classify sentiments without task-specific fine-tuning.
- **Multimodal Sentiment Analysis:** Combines text with images or audio for better sentiment understanding.
- **Explainability:** Focus on making model predictions interpretable.

WHAT IS PYTORCH

PyTorch is an open-source deep learning framework that's known for its flexibility and ease-of-use. This is enabled in part by its compatibility with the popular Python high-level programming language favored by machine learning developers and data scientists.

WHY PYTORCH

Image reference https://pytorch.org/features/

PyTorch is a fully featured framework for building deep learning models, which is a type of machine learning that's commonly used in applications like image recognition and language processing.

THE PYTORCH EXPLAINED

PYTORCH TENSORS

The Pytorch is used to process the tensors. Tensors are multidimensional arrays like n-dimensional NumPy array. However, tensors can be used in GPUs as well, which is not in the case of NumPy array. PyTorch accelerates the scientific computation of tensors as it has various inbuilt functions.

A vector is a one-dimensional tensor, and a matrix is a two-dimensional tensor. One significant difference between the Tensor and multidimensional array used in C, C++, and Java is tensors should have the same size of columns in all dimensions. Also, the tensors can contain only numeric data types.

VECTOR 1 D TENSOR

1.0
2.0
3.0

MATRIX 2 D TENSOR

1.0	4.0	7.0
2.0	5.0	8.0
3.0	6.0	9.0

The two fundamental attributes of a tensor are:

- **Shape** refers to the dimensionality of array or matrix
- **Rank** refers to the number of dimensions present in tensor

Code	Output

# importing torch 	
import torch
 	
# creating a tensors 	
t1=torch.tensor([1, 2, 3, 4])
t2=torch.tensor([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
 	
# printing the tensors: 	
print("Tensor t1: \n", t1)
print("\nTensor t2: \n", t2)
 	
# rank of tensors 	
print("\nRank of t1: ", len(t1.shape))
print("Rank of t2: ", len(t2.shape))
 	
# shape of tensors 	
print("\nRank of t1: ", t1.shape)
print("Rank of t2: ", t2.shape)
	Tensor t1:
	tensor ([1, 2, 3, 4])
	Tensor t2:
	tensor([[1, 2, 3, 4], [5,6,7,8], [9,10,11,12]])
	Rank of t1: 1
	Rank of t2: 2
	Rank of t1: torch.Size([4])
	Rank of t2: torch.Size([3,4])

CREATING TENSOR IN PYTORCH

There are various methods to create a tensor in PyTorch. A tensor can contain elements of a single data type. We can create a tensor using a python list or NumPy array. The torch has 10 variants of tensors for both GPU and CPU. Below are different ways of defining a tensor.

torch.Tensor()
It copies the data and creates its tensor. It is an alias for torch.FloatTensor.

torch.tensor()
It also copies the data to create a tensor; however, it infers the data type automatically.

torch.as_tensor()
The data is shared and not copied in this case while creating the data and accepts any type of array for tensor creation.

torch.from_numpy()
It is similar to tensor.as_tensor() however it accepts only numpy array.

Code	Output

# importing torch module 	
import torch
import numpy as np
 	
# list of values to be stored as tensor 	
data1 = [1, 2, 3, 4, 5, 6]
data2 = np.array([1.5, 3.4, 6.8, 9.3, 7.0, 2.8])
 	
# creating tensors and printing 	
t1 = torch.tensor(data1)
t2 = torch.Tensor(data1)
t3 = torch.as_tensor(data2)
t4 = torch.from_numpy(data2)
 	
print("Tensor: ",t1, "Data type: ", t1.dtype,"\n")
print("Tensor: ",t2, "Data type: ", t2.dtype,"\n")
print("Tensor: ",t3, "Data type: ", t3.dtype,"\n")
print("Tensor: ",t4, "Data type: ", t4.dtype,"\n")
	Tensor: tensor ([1, 2, 3, 4, 5, 6])
	Data type: torch.int64
	Tensor: tensor ([1., 2., 3., 4., 5., 6.])
	Data type: torch.float32
	Tensor: tensor([1.5000, 3.4000, 6.8000, 9.3000, 7.0000, 2.80e0], dtype=torch.float64)
	Data type: torch. float64
	Tensor: tensor ([1.5000, 3.4000, 6.8000, 9.3000, 7.0000, 2.800e], dtype=torch.float64)
	Data type: torch. float64

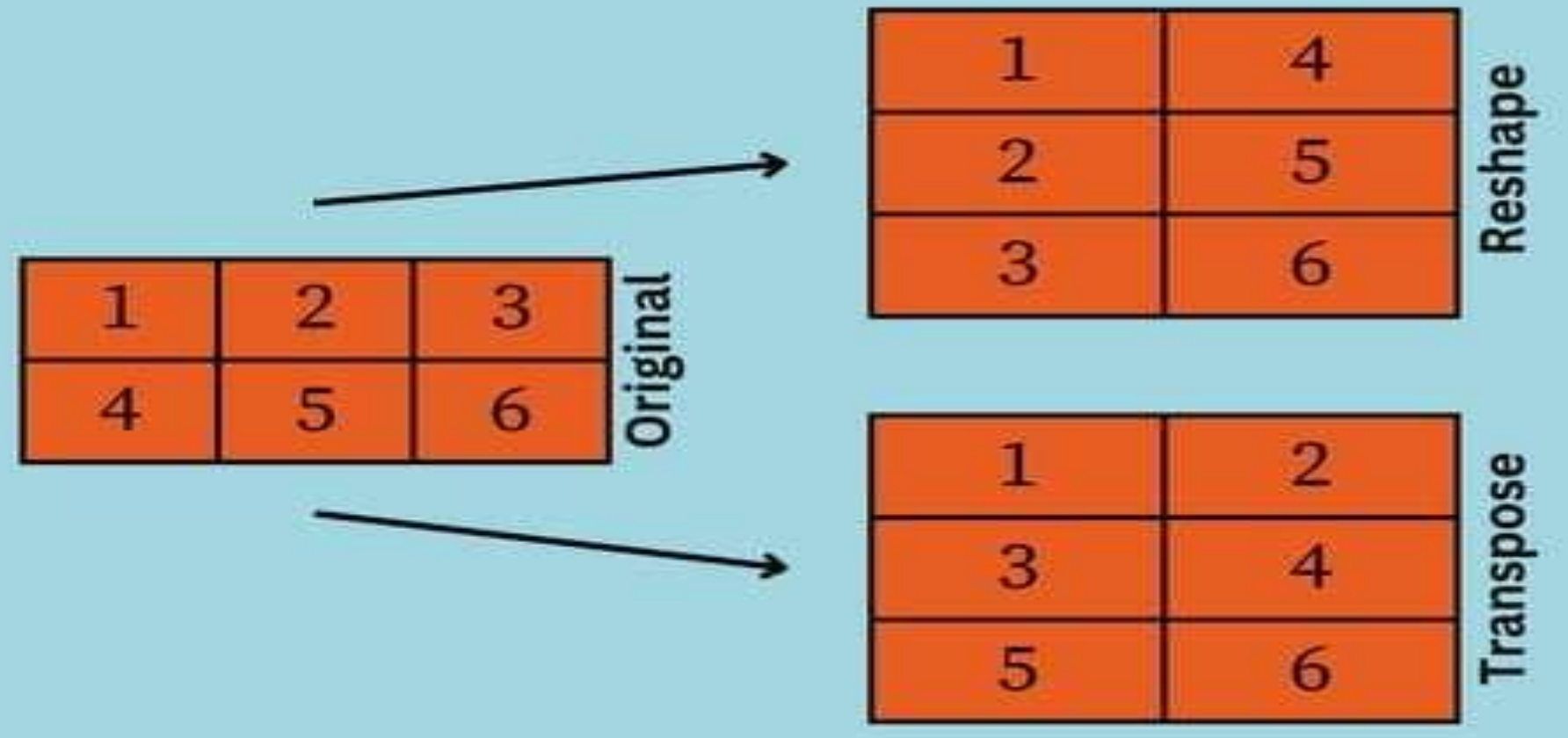
RESTRUCTURING TENSORS IN PYTORCH

We can modify the shape and size of a tensor as desired in PyTorch. We can also create a transpose of an n-d tensor. Below are three common ways to change the structure of your tensor as desired:

.reshape(a, b)
returns a new tensor with size a,b

.resize(a, b)
returns the same tensor with the size a,b

.transpose(a, b)
returns a tensor transposed in a and b dimension



Reshape and Transpose Difference

A 2*3 matrix has been reshaped and transposed to 3*2. We can visualize the change in the arrangement of the elements in the tensor in both cases.

Code	Output

# import torch module 	
import torch
 	
# defining tensor 	
t = torch.tensor([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
 	
# reshaping the tensor 	
print("Reshaping")
print(t.reshape(6, 2))
 	
# resizing the tensor 	
print("\nResizing")
print(t.resize(2, 6))
 	
# transposing the tensor 	
print("\nTransposing")
print(t.transpose(1, 0))
	Reshaping
	tensor([[1, 2], [3, 4], [5, 6], [7, 8], [9, 10], [11, 12]])
	
	Resizing
	tensor ([[1, 2, 3, 4, 5, 6], [7, 8, 9, 10, 11, 12]])
	
	Transposing
	tensor([[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]])

MATHEMATICAL OPERATIONS ON TENSORS USING PYTORCH

We can perform various mathematical operations on tensors using Pytorch. The code for performing Mathematical operations is the same as in the case with NumPy arrays. Below is the code for performing the four basic operations in tensors:

CODE

```
# import torch module
import torch
 # defining two tensors
t1 = torch.tensor([1, 2, 3, 4])
t2 = torch.tensor([5, 6, 7, 8])
 # adding two tensors
print("tensor2 + tensor1")
print(torch.add(t2, t1))
 # subtracting two tensor
print("\ntensor2 - tensor1")
print(torch.sub(t2, t1))
 # multiplying two tensors
print("\ntensor2 * tensor1")
print(torch.mul(t2, t1))
 # diving two tensors
print("\ntensor2 / tensor1")
print(torch.div(t2, t1))
```

OUTPUT

```
Tensor: tensor ([1, 2, 3, 4, 5, 6])
Data type: torch.int64
Tensor: tensor ([1., 2., 3., 4., 5., 6.])
Data type: torch.float32
Tensor: tensor([1.5000, 3.4000, 6.8000, 9.3000,
7.0000, 2.80e0], dtype=torch.float64)
Data type: torch. float64
Tensor: tensor ([1.5000, 3.4000, 6.8000, 9.3000,
7.0000, 2.800e], dtype=torch.float64)
Data type: torch. float64
```

PYTORCH MODULES

The PyTorch library modules are essential to create and train neural networks. The three main library modules are Autograd, Optim, and nn.

 # 1. Autograd Module

The autograd provides the functionality of easy calculation of gradients without the explicitly manual implementation of forward and backward pass for all layers. For training any neural network we perform backpropagation to calculate the gradient. By calling the .backward() function we can calculate every gradient from the root to the leaf.

 # 2. Optim Module

PyTorch Optium Module which helps in the implementation of various optimization algorithms. This package contains the most commonly used algorithms like Adam, SGD, and RMS-Prop. To use torch.optim we first need to construct an Optimizer object which will keep the parameters and update it accordingly. First, we define the Optimizer by providing the optimizer algorithm we want to use. We set the gradients to zero before backpropagation. Then for updation of parameters the optimizer.step() is called.

PYTORCH MODULES

 # 3. nn Module

This package helps in the construction of neural networks. It is used to build layers.

For creating a model with a single layer we can simply define it by using nn.Sequential().

For the implementation of the model which is not in a single sequence, we define a model by subclassing the nn.Module class.

PYTORCH VS TENSORFLOW

PyTorch	TensorFlow
PyTorch is closely related to the lua-based Torch framework which is actively used in Facebook.	TensorFlow is developed by Google Brain and actively used at Google.
PyTorch is relatively new compared to other competitive technologies.	TensorFlow is not new and is considered as a to-go tool by many researchers and industry professionals.
PyTorch includes everything in imperative and dynamic manner.	TensorFlow includes static and dynamic graphs as a combination.
Computation graph in PyTorch is defined during runtime.	TensorFlow do not include any run time option.

KEY FEATURES OF USING PYTORCH

TENSOR COMPUTATION

Tensors are generic n-dimensional arrays used for arbitrary numeric computation and are accelerated by graphics processing units. These multidimensional structures can be operated on and manipulated with application program interfaces (APIs).

TORCHSCRIPT

This is the production environment of PyTorch that enables users to seamlessly transition between modes. TorchScript optimizes functionality, speed, ease of use and flexibility.

AUTOMATIC DIFFERENTIATION

This technique is used for creating and training neural networks. It numerically computes the derivative of a function by making backward passes in neural networks.

KEY FEATURES OF USING PYTORCH

DYNAMIC GRAPH COMPUTATION

This feature lets users change network behavior on the fly, rather than waiting for all the code to be executed.

PYTHON SUPPORT

Because PyTorch is based on Python, it can be used with popular libraries and packages such as NumPy, SciPy, Numba and Cynthon.

PYTORCH BENIFITS

- Offers developers an easy-to-learn, simple-to-code structure that's based on Python.
- Enables easy debugging with popular Python tools.
- Offers scalability and is well-supported on major cloud platforms.
- Provides a small community focused on open source.
- Exports learning models to the Open Neural Network Exchange (ONNX) standard format.
- Offers a user-friendly interface.
- Provides a C++ front-end interface option.
- Includes a rich set of powerful APIs that extend the PyTorch library.

MEMBERS AND GOVERNING BOARD

AMD

aws

Meta

Google Cloud

Microsoft Azure

NVIDIA

PYTORCH USE CASES

- Natural Language Processing (NLP): From Siri to Google Translate, deep neural networks have enabled breakthroughs in machine understanding of natural language. Most of these models treat language as a flat sequence of words or characters and use a kind of model called a recurrent neural network (RNN) to process this sequence. But many linguists think that language is best understood as a hierarchical tree of phrases, so a significant amount of research has gone into deep learning models known as recursive neural networks that take this structure into account. While these models are notoriously hard to implement and inefficient to run, PyTorch makes these and other complex natural language processing models a lot easier. Salesforce is using PyTorch for NLP and multi-task learning.

PYTORCH USE CASES

- Research: PyTorch is a popular choice for research due to its ease of use, flexibility and rapid prototyping. Stanford University is using PyTorch's flexibility to efficiently research new algorithmic approaches.

- Education: Udacity is educating AI innovators using PyTorch.